

단계1: docker build

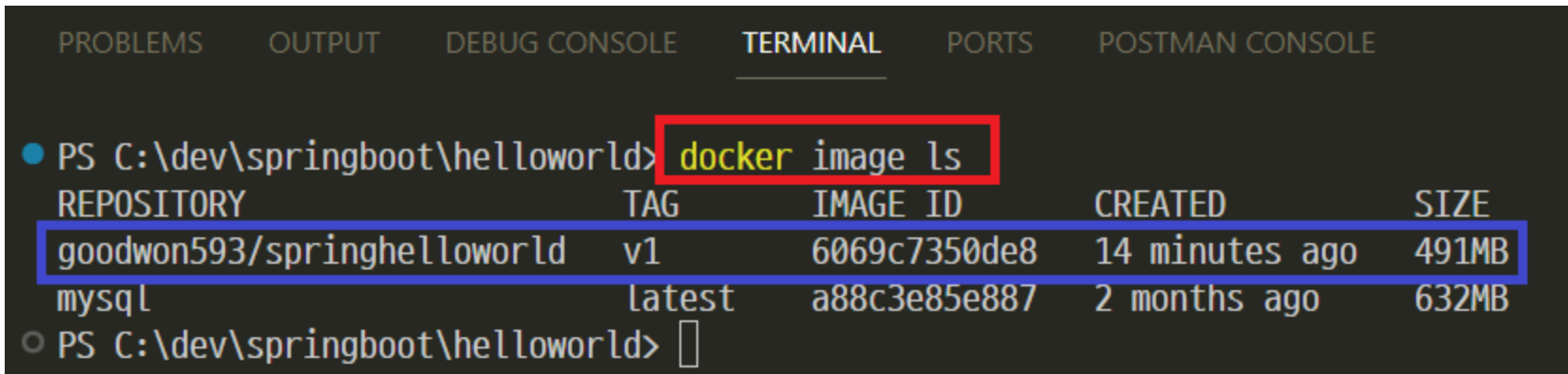
```
docker build -t goodwon593/springhelloworld:v1 .
```

```
PS C:\dev\springboot\helloworld> docker build -t goodwon593/springhelloworld:v1 .
[+] Building 1.6s (19/19) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 647B
=> [internal] load metadata for docker.io/library/openjdk:17.0.2-jdk
=> [internal] load metadata for docker.io/library/gradle:7.6-jdk17-alpine
=> [auth] library/openjdk:pull token for registry-1.docker.io
=> [auth] library/gradle:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [build 1/8] FROM docker.io/library/gradle:7.6-jdk17-alpine@sha256:ef0422e948f93a276e90de8d5619fd6c090d5ba344918ecb8142f24f7fe9a15f
=> [internal] load build context
=> => transferring context: 1.21kB
=> [stage-1 1/3] FROM docker.io/library/openjdk:17.0.2-jdk@sha256:528707081fdb9562eb819128a9f85ae7fe000e2fbaef9f87662e7b3f38cb7d8
=> CACHED [stage-1 2/3] WORKDIR /apps
• => CACHED [build 2/8] WORKDIR /apps
=> CACHED [build 3/8] COPY build.gradle settings.gradle gradlew /apps
=> CACHED [build 4/8] COPY gradle /apps/gradle
=> CACHED [build 5/8] RUN chmod +x gradlew
=> CACHED [build 6/8] RUN ./gradlew build || return 0
=> CACHED [build 7/8] COPY src /apps/src
=> CACHED [build 8/8] RUN ./gradlew clean build
=> CACHED [stage-1 3/3] COPY --from=build /apps/build/libs/helloworld-0.0.1-SNAPSHOT.jar app.jar
=> exporting to image
=> => exporting layers
=> => writing image sha256:6069c7350de8c7518d85262365a746b71058daa71cd914f2c0b666f20b17f71b
=> => naming to docker.io/goodwon593/springhelloworld:v1

What's Next?
View a summary of image vulnerabilities and recommendations → docker scout quickview
○ PS C:\dev\springboot\helloworld> 
```

단계2: docker image 확인

```
docker image ls
```



```
PS C:\dev\springboot\helloworld> docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
goodwon593/springhelloworld	v1	6069c7350de8	14 minutes ago	491MB
mysql	latest	a88c3e85e887	2 months ago	632MB

```
PS C:\dev\springboot\helloworld>
```

단계3: docker container

```
docker run --name spring-helloworld -d -p 8080:8080 goodwon593/springhelloworld:v1
```

```
PS C:\dev\springboot\helloworld> docker run --name spring-helloworld -d -p 8080:8080 goodwon593/springhelloworld:v1
38991effa14a927a5653afd1226a59377b74b55868b8b8c08abb9f21b0ae2d89
PS C:\dev\springboot\helloworld>
```

The screenshot shows the Docker Desktop application window. The 'Containers' tab is active, displaying a table of running containers. A red arrow points from the terminal command to the 'spring-helloworld' container entry. The container's details are highlighted with a blue box.

Name	Image	Status	CPU (%)	Port(s)	Last started	Actions
spring-helloworld 38991effa14a	goodwon593/springhelloworld:v1	Running	0%	8080:8080	23 seconds ago	Stop, Restart, Delete
> mysql		Running (1/1)	0.64%		1 hour ago	Stop, Restart, Delete

단계4: docker container 확인

```
docker ps
```

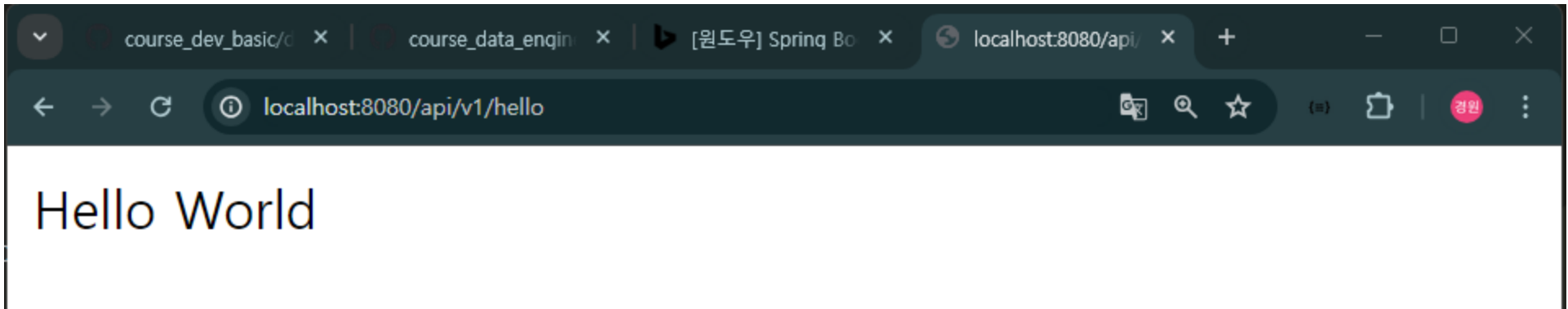
```
PS C:\dev\springboot\helloworld> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
38991effa14a	goodwon593/springhelloworld:v1	"java -jar app.jar"	5 minutes ago	Up 5 minutes	0.0.0.0:8080->8080/tcp	spring-helloworld
18e3a5e2ce07	mysql	"docker-entrypoint.s..."	3 weeks ago	Up 2 hours	0.0.0.0:3306->3306/tcp, 33060/tcp	mysql-db-1

```
PS C:\dev\springboot\helloworld>
```

단계5: 테스트

- <http://localhost:8080/api/v1/hello>



참고자료

- <https://sangchul.kr/entry/윈도우-Spring-Boot-프로젝트-Gradle로-Docker-이미지-빌드하기>